

Ellenfél alapú támadások (FGSM, PGD)  
implementációja  
konvolúciós neurális hálózatokhoz  
interaktív tesztkörnyezetben

Gabrity Gábor  
Neptun kód: D9V09Z  
2026. szeptember 15.

# Nyilatkozat

Alulírott Gabrity Gábor (Neptun kód: D9V09Z), az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférést a benyújtástól számított két évig nem kérem.

Budapest, 2026. szeptember 15.

Gabrity Gábor s.k.  
hallgató aláírása

## Kivonat

# Tartalomjegyzék

|                                                                   |          |
|-------------------------------------------------------------------|----------|
| <b>1. Rövidítés- és jelölésjegyzék</b>                            | <b>3</b> |
| <b>2. Bevezetés</b>                                               | <b>4</b> |
| 2.1. Motiváció és a témaválasztás indoklása . . . . .             | 4        |
| 2.2. A megoldandó feladat rövid, közérthető leírása . . . . .     | 4        |
| 2.3. A dolgozat célja és hozzájárulásai . . . . .                 | 4        |
| 2.4. A dolgozat felépítése . . . . .                              | 4        |
| <b>3. Felhasználói dokumentáció</b>                               | <b>5</b> |
| 3.1. A megoldott probléma rövid megfogalmazása . . . . .          | 5        |
| 3.2. A felhasznált módszerek rövid áttekintése . . . . .          | 5        |
| 3.3. Rendszerkövetelmények . . . . .                              | 5        |
| 3.4. Telepítés és első indítás . . . . .                          | 5        |
| 3.5. A grafikus felhasználói felület bemutatása . . . . .         | 5        |
| 3.6. Tipikus felhasználási forgatókönyvek . . . . .               | 5        |
| 3.7. Hibaüzenetek és gyakori problémák . . . . .                  | 5        |
| <b>4. Fejlesztői dokumentáció</b>                                 | <b>6</b> |
| 4.1. A probléma részletes specifikációja . . . . .                | 6        |
| 4.1.1. A felhasználói funkciók vázlatos áttekintése . . . . .     | 6        |
| 4.2. Elméleti háttér és a használt fogalmak definíciója . . . . . | 7        |
| 4.3. Ellenfél alapú támadások áttekintése . . . . .               | 7        |
| 4.4. FGSM (Fast Gradient Sign Method) . . . . .                   | 7        |
| 4.5. PGD (Projected Gradient Descent) . . . . .                   | 7        |
| 4.5.1. Hiperparaméterek hatása . . . . .                          | 7        |
| 4.5.2. Előnyök és korlátok . . . . .                              | 8        |
| 4.6. Tervezési döntések és indoklásuk . . . . .                   | 8        |
| 4.7. Kapcsolódó munkák és további támadási módszerek . . . . .    | 8        |
| 4.8. Védekezési stratégiák áttekintése . . . . .                  | 8        |
| 4.9. A rendszer architektúrája . . . . .                          | 8        |
| 4.10. A felhasznált technológiák . . . . .                        | 8        |
| 4.11. Az egyes komponensek megvalósítása . . . . .                | 8        |
| <b>5. Tesztelés</b>                                               | <b>9</b> |
| 5.1. Egységtesztek . . . . .                                      | 9        |
| 5.2. Integrációs tesztek . . . . .                                | 12       |
| 5.2.1. GUI és támadási motor összekötése . . . . .                | 12       |
| 5.3. Automatizált GUI tesztek . . . . .                           | 13       |
| 5.4. Funkcionális tesztek és felhasználói validáció . . . . .     | 13       |

|                                                |           |
|------------------------------------------------|-----------|
| 5.5. Teszteredmények és értelmezésük . . . . . | 15        |
| 5.6. A tesztelés korlátai . . . . .            | 15        |
| <b>6. Összefoglalás</b>                        | <b>16</b> |
| 6.1. Elért eredmények . . . . .                | 16        |
| 6.2. Következtetések . . . . .                 | 16        |
| 6.3. Továbbfejlesztési lehetőségek . . . . .   | 16        |
| <b>Mellékletek</b>                             | <b>17</b> |

## 1. Rövidítés- és jelölésjegyzék

## **2. Bevezetés**

**2.1. Motiváció és a témaválasztás indoklása**

**2.2. A megoldandó feladat rövid, közérthető leírása**

**2.3. A dolgozat célja és hozzájárulásai**

**2.4. A dolgozat felépítése**

### **3. Felhasználói dokumentáció**

**3.1. A megoldott probléma rövid megfogalmazása**

**3.2. A felhasznált módszerek rövid áttekintése**

**3.3. Rendszerkövetelmények**

**3.4. Telepítés és első indítás**

**3.5. A grafikus felhasználói felület bemutatása**

**3.6. Tipikus felhasználási forgatókönyvek**

**3.7. Hibaiüzenetek és gyakori problémák**

## 4. Fejlesztői dokumentáció

### 4.1. A probléma részletes specifikációja

Ebben a fejezetben szeretném a megvalósítandó szoftver célját és funkcióit definiálni. Legyen adott egy  $f_\theta$  neurális háló alapú osztályozó modell, melynek  $x$  egy bemenete és  $y$  annak helyes címkéje. Az  $x$  bemenet ebben az esetben egy kép pixeleinek valamely három színű kódolása, vektoriális formában, az  $y$  pedig a kép helyes osztályának indexe, például  $y = 42$  az agámák osztályát reprezentálja. Amennyiben a képhez tartozó  $y$  helyes címke nem ismert, úgy a modell által meghatározott legvalószínűbb címkét  $\hat{y}$ -nal jelöljük, és ezt használjuk referenciacímkeként. Célunk az  $x$  képre egy  $x_{\text{adv}}$  bemenet előállítása, mely az eredeti  $x$ -től szemmel csak alig látható perturbációban különbözik, viszont a modell által adott  $\hat{y}_{\text{adv}} = \arg \max_i f_\theta(x_{\text{adv}})_i$  predikció a referenciaosztálytól eltér, vagy a klasszifikáció elbizonytalanodik. Alapvetően két támadási módot különböztetünk meg, az ún. targeted és non-targeted módot. Non-targeted esetben a támadás célja minél jobban elbizonytalanítani a helyes osztályozást, tehát hogy agáma helyett bármi mást mutasson. Targeted módban azt szeretnénk, hogy egy kijelölt  $y_{\text{target}}$  osztály irányába változzon a képünk. (pl. komodói varánusz) A támadás feltétele, hogy az  $f_\theta$  modell gradienseihez és paramétereire teljes hozzáférésünk legyen. Ezt white-box hozzáférésnek nevezzük AI red teaming kontextusban.

E szoftver célja a fenti ellenfél alapú támadó tesztkörnyezet szoftveres megvalósítása, különböző támadási algoritmusokkal (FGSM, PGD). A szoftver a PyTorch (torchvision) előtanított modelljeire épül; az architektúra tetszőleges torchvision-moddal bővíthető, jelenleg kettő van integrálva (ResNet50, MobileNetV2). Beállíthatóak a futás paraméterei:

- modell: ResNet50 vagy MobileNetV2, amelyen a támadást végrehajtjuk,
- $\epsilon$ , mely meghatározza a perturbáció maximális mértékét, tehát azt, hogy egy pixelen legfeljebb mekkora változást tapasztalhatunk, azaz a változás felső határát,
- cél osztály,  $y_{\text{target}}$ , ha a mód targeted,
- iterációszám (PGD esetén hány lépésig engedjük futtatni az algoritmust).

#### 4.1.1. A felhasználói funkciók vázlatos áttekintése

A fenti specifikációt a szoftver az alábbi, a felhasználó számára közvetlenül elérhető funkciókkal valósítja meg:

- a fájlrendszerrel kiválasztott képet legördülő menüből kiválasztott modellel klasszifikáljuk,
- támadási algoritmus és paraméterek meghatározásával perturbációt és adversarial képet generálunk,

- megjelenítjük az eredeti képet, a perturbációt és az adversarial képet, továbbá mindkét (eredeti és adversarial) képet osztályozzuk is,
- a védekezési modulban JPEG-tömörítést vagy más képfeldolgozási szűrőt alkalmazhatunk az adversarial perturbáció hatásának utólagos csökkentésére vagy semlegesítésére.

## 4.2. Elméleti háttér és a használt fogalmak definíciója

Jelöljük  $f_\theta(x) = z$ -vel a modell outputjaként előállt valós számok tömbjét (*logit* vektort), ahol  $z \in \mathbb{R}^C$  és  $C$  az osztályok száma. (ImageNet esetén  $C = 1000$ ) Az elnevezés a logisztikus regresszióból származik, ahol a  $\text{logit}(p) = \ln \frac{p}{1-p}$  függvény egy valószínűséget képez le a valós számegyenesre. A  $z_i$  komponensek páronkénti különbsége éppen az  $i$ -edik és  $j$ -edik osztály esélyhányadosának logaritmusával egyezik meg,  $z_i - z_j = \ln \frac{p_i}{p_j}$ , innen ez az elnevezés.

Mivel a  $z$  logitok önmagukban nem valószínűségek, a softmax függvénnyel alakítjuk át őket valószínűségi eloszlássá:

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad i = 1, \dots, C.$$

Tehát  $p_i$  a modell szerint annak a valószínűsége, hogy  $x$  az  $i$ -edik osztályba tartozik. A modell által meghatározott legvalószínűbb osztály indexét a legnagyobb valószínűségű komponens kiválasztásával kapjuk:

$$\hat{y} = \arg \max_i p_i.$$

## 4.3. Ellenfél alapú támadások áttekintése

### 4.4. FGSM (Fast Gradient Sign Method)

### 4.5. PGD (Projected Gradient Descent)

#### 4.5.1. Hiperparaméterek hatása

A PGD viselkedését három fő hiperparaméter határozza meg:

- $\epsilon$ : a megengedett maximális perturbáció. Nagyobb  $\epsilon$  erősebb támadást tesz lehetővé, de a módosítás emberi szemmel is láthatóvá válhat.
- $\alpha$ : a lépésnagyság. Túl kis  $\alpha$  lassú konvergenciát eredményez, túl nagy  $\alpha$  pedig instabilitáshoz vezethet. Tipikus választás:  $\alpha = \epsilon/T$  vagy  $\alpha = 2.5 \cdot \epsilon/T$ .
- $T$ : az iterációk száma. Több iteráció általában erősebb támadást eredményez, de növeli a számítási költséget.

#### **4.5.2. Előnyök és korlátok**

A PGD az FGSM-nél lényegesen erősebb támadásokat képes generálni, mivel több kis lépésben közelíti a lokális optimumot. A random restart technikával kombinálva különösen hatékony. Hátránya, hogy  $T$ -szer annyi gradiens-számítást igényel, mint az FGSM, így nagyobb a számítási költsége.

#### **4.6. Tervezési döntések és indoklásuk**

#### **4.7. Kapcsolódó munkák és további támadási módszerek**

#### **4.8. Védekezési stratégiák áttekintése**

#### **4.9. A rendszer architektúrája**

#### **4.10. A felhasznált technológiák**

#### **4.11. Az egyes komponensek megvalósítása**

## 5. Tesztelés

### 5.1. Egységtesztek

A program tesztkészletének felépítésénél a szakirodalomból ismert *tesztpiramisnak* nevezett hierarchiát követtem [3]. Ennek alapját a gyorsabb és kevés komponenst érintő tesztek, az egységtesztek képezik, melyek a lehető legkisebb elkülöníthető egységek tesztelését valósítják meg. Futási idejük jellemzően ezredmásodperces tartományban mozog és akár lefedhetik az input paraméterek teljes értékkészletét. Az elkészítésüknél fontos szempont a többi egységtől való függetlenség (izoláció) és az perifériáktól való függetlenség, I/O, adatbázis, hálózat, stb. Ezeknek a teszteknek az elkészítésére érdemes nagy figyelmet fordítani, mivel a korai szakaszban feltárt hibák javítása általában lényegesen olcsóbb, mint a későbbi, integráció vagy éles környezetben felfedezetté.

Az egységtesztek tudatosan nem a valódi, előtanított ResNet50/MobileNetV2 modelleket használják. Az `attack_engine` teszteknel egy `tiny_model` fixture ad egy 5 osztályos,  $8 \times 8$  pixeles bemenetet feldolgozó, egyetlen lineáris rétegből álló hálót. Ez a döntés a tesztek sebességét (a valódi háló letöltése és felépítése másodperces nagyságrendű lenne) és izoláltságát szolgálja: a teszt kizárólag a saját kódunk logikáját vizsgálja, nem a harmadik féltől származó architektúra pontosságát.

Minden véletlent felhasználó fixture (pl. a bemeneti képet generáló `sample_image_tensor`) maga hívja meg a `torch.manual_seed`-et közvetlenül az adat előállítása előtt, így a determinizmus független a fixture-ök végrehajtási sorrendjétől. Az egységtesztek szintjén nem támadási sikerrátát mérünk (ehhez sok mintás, statisztikai kiértékelés kellene), hanem az algoritmusok tulajdonságait ellenőrizzük konkrét, kis méretű bemeneteken.

Ebben a részben a számosságuk miatt nem mutatom be az összes egységtesztet, csak a középpontban álló `attack_engine.py` modult mivel ez valósítja meg az alkalmazás lényegi logikáját (az FGSM és PGD algoritmusokat, valamint az ezeket kiszolgáló `predict()` és `attack()` függvényeket).

#### A `predict()` és a diszpécser (`attack()`) tesztjei

`test_predict_returns_topk_shapes`: Ellenőrzi, hogy `topk=3` esetén a visszaadott valószínűség- és index-tenzor alakja  $(1, 3)$ , azaz a batch- és  $k$ -dimenzió megfelel a kérésnek.

---

`test_predict_returns_valid_distribution`: A `topk=5` valószínűségek mindegyike a  $[0, 1]$  intervallumba esik, és – mivel az 5 osztályos `tiny_model` minden osztálya lekérdezésre kerül – összegük 1,0-ra `allclose`. Ez a softmax kimenet valódi valószínűségi eloszlás voltát igazolja.

---

`test_predict_topk_returns_highest_first`: A visszaadott valószínűségek csökkenő sorrendben szerepelnek ( $\text{probs}[0, i] \geq \text{probs}[0, i + 1]$ ), vagyis a `topk` valóban a legnagyobb konfidenciájú osztályokat adja vissza, még hozzá rendezetten.

---

`test_attack_unknown_method_raises`: Határeset: egy nem létező módszernév ("NEM VALID") megadásakor `ValueError`-t vár, mielőtt bármilyen gradiensszámítás történne – a bemenet-validáció a drága számítás előtt fut le.

---

`test_attack_fgsm_ignores_iterations_argument`: Annak ellenére, hogy a hívó `iterations=99`-et ad át, FGSM esetén a visszaadott `AttackResult.iterations_run` értéke 1 – igazolva, hogy az FGSM valóban egylépéses, és a paramétert a diszpécser figyelmen kívül hagyja erre a módszerre.

---

`test_attack_pgd_reports_requested_iteration_count`: PGD esetén a kért iterációszám (4) változatlanul megjelenik az `iterations_run` mezőben, alátámasztva, hogy a diszpécser a paramétert továbbadja a `_pgd` implementációnak.

---

`test_attack_fgsm_dispatches_only_to_fgsm`: `mock.patch(..., wraps=_fgsm)` és `wraps=_pgd` spy-okkal figyeli a hívásokat: `method="FGSM"` esetén `_fgsm.assert_called_once()` és `_pgd.assert_not_called()` teljesül. A `wraps` paraméter miatt a valódi implementáció is lefut, tehát az `AttackResult` összeállítása is ellenőrzött marad.

---

`test_attack_pgd_dispatches_only_to_pgd`: Az előző teszt tükörképe: `method="PGD"` esetén kizárólag `_pgd` hívódik meg, `_fgsm` nem. A két teszt együtt zárja ki, hogy a diszpécser valamelyik ágon „átcsorogva” mindkét algoritmust lefuttassa.

### Az FGSM implementáció (`_fgsm`) tesztjei

`test_fgsm_targeted_steps_opposite_direction_of_untargeted`: Azonos bemeneten és  $\epsilon$  mellett a `targeted` és `non-targeted` perturbáció pontosan ellentétes előjelű ( $x_{\text{untargeted}} - x = -(x_{\text{targeted}} - x)$ ), ami közvetlenül a matematikai definícióból (a gradiens előjelének megfordításából) következik.

---

`test_fgsm_clamps_to_valid_pixel_range`: Szélsőséges bemenet (csupa nulla pixel) és nagy  $\epsilon = 0,5$  mellett is a kimenet minden eleme a  $[0, 1]$  tartományban marad – igazolva, hogy a `clamp` lépés ténylegesen érvényesül, nem csak elméletben szerepel a specifikációban.

---

`test_fgsm_result_is_detached_from_autograd`: A visszaadott tenzor `requires_grad` attribútuma `False`.

---

`test_fgsm_result_has_no_graph_history`: Szigorúbb bizonyíték az előzőnél: `grad_fn` `is None` és `is_leaf` `is True` egyszerre teljesül, ami kizárja azt az esetet is, amikor egy tenzor `requires_grad` értéke hamis, de mégis egy számítási gráf levele marad rejtett hivatkozásokkal.

---

`test_fgsm_zero_epsilon_returns_original`: Határeset:  $\epsilon = 0$  esetén a perturbáció nulla, a kimenet `allclose` az eredeti képpel – az algoritmus degenerált esetben identitásként viselkedik.

---

`test_fgsm_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték: a `predict()`-en keresztül lekérdezett, a valódi címkéhez tartozó konfidencia a támadás után szigorúan kisebb, mint előtte – ez nem csak a perturbáció létezését, hanem annak tényleges hatását igazolja a modell kimenetére.

---

`test_fgsm_targeted_increases_confidence_in_target_label`: A `targeted` változat tükörpárja: a mesterségesen kiválasztott célosztály konfidenciája nő a támadás hatására, alátámasztva, hogy a `targeted` mód valóban a célosztály felé tereli a predikciót, nem csupán „valamerre” téveszti meg a modellt.

## A PGD implementáció (`_pgd`) tesztjei

`test_pgd_result_stays_within_epsilon_ball`: Mérsékelt lépésméret ( $\alpha = 0,02$ ) mellett a végső perturbáció minden pixelre  $|\delta| \leq \epsilon + 10^{-6}$  (a tolerancia a lebegőpontos kerekítést kompenzálja) – ez az  $\ell_\infty$ -gömbbe való projekció alapvető helyességét igazolja.

---

`test_pgd_hits_ball_when_step_too_large`: Szándékosan irreálisan nagy lépésméretet ( $\alpha = 10,0 \gg \epsilon = 0,02$ ) ad meg, ami minden iterációban túllövést okoz; ilyenkor a projekciós `clamp`-nek kell egyedül biztosítania a korlátot, ezért a tesztben a végső  $|\delta|$  pontosan (nem csak  $\leq$ )  $\epsilon$ -nak adódik minden pixelen – ez a projekció, nem a lépésméret helyes megválasztása általi korlátozottságot bizonyítja.

---

`test_pgd_clamps_to_valid_pixel_range`: A pixeltartomány-korlátozás ugyanúgy teljesül PGD esetén is, mint FGSM-nél: szélsőséges (nulla) bemeneten és nagy  $\epsilon$  mellett a kimenet a  $[0, 1]$  tartományban marad.

---

`test_pgd_result_has_no_graph_history`: Ugyanaz az erős detach-bizonyíték (`grad_fn is None` és `is_leaf is True`), mint FGSM esetén – fontos, mert a PGD több iterációt is végez, minden lépésben új `requires_grad_(True)` hívással, így könnyebben „elszivároghatna” egy hivatkozás a gráfra, ha a ciklus végén elmaradna a leválasztás.

---

`test_pgd_zero_epsilon_returns_original`: Határeset:  $\epsilon = 0$  esetén – annak ellenére, hogy több iteráció is lefut – a projekció minden lépésben nulla szélességű intervallumra szorítja a perturbációt, így a végeredmény megegyezik az eredeti képpel.

---

`test_pgd_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték `predict()`-en keresztül: a valódi címke konfidenciája több iteráció után is csökken – ugyanaz az elv, mint FGSM-nél, de itt azt is igazolja, hogy az iteratív frissítés nem „rontja el” a kumulatív hatást.

---

`test_pgd_targeted_increases_confidence_in_target_label`: A targeted PGD a cél-osztály konfidenciáját növeli – ugyanazon tulajdonságpár (non-targeted csökkenti/targeted növeli a releváns konfidenciát) FGSM-re és PGD-re is parametrizált formában lett tesztelve (`@pytest.mark.parametrize`), elkerülve, hogy csak az egyik algoritmusra íródjon le a viselkedés.

---

## 5.2. Integrációs tesztek

### 5.2.1. GUI és támadási motor összekötése

Az integrációs réteg feladata annak ellenőrzése, hogy az `app_interface.py` eseménykezelői helyesen építik-e fel az `AttackContext`-et a ténylegesen betöltött `ModelBundle`-ból, és a `attack_engine.attack()` hívás eredménye helyesen jelenik-e meg a felületen (kép- és predikció-frissítés). Ez a réteg már nem mockolja a modellt, de a valódi felhasználói interakciót (egérekattintás, csúszka-mozgatás) sem szimulálja még – azt a GUI-tesztek rétege végzi.

### 5.3. Automatizált GUI tesztek

A GUI tesztek a felhasználói felületet magát, a ténylegesen megjelenő ablakot és widgeteket vizsgálják, programozott módon szimulálva a felhasználói interakciókat, majd ellenőrizi, hogy a felület a várt módon reagál-e. Ez a legfelső, egyben legköltségesebb és leglassabb réteg a tesztpiramisban, hiszen minden egyes esethez fel kell építeni a teljes grafikus felületet – ezért csak kevés, a legfontosabb felhasználói forgatókönyveket lefedő tesztesetre érdemes szorítkozni, a részletesebb logikai ellenőrzést az alsóbb rétegekre (egység- és integrációs tesztek) hagyva.

### 5.4. Funkcionális tesztek és felhasználói validáció

A grafikus felület funkcionális, „fekete doboz” tesztéseit a felhasználói forgatókönyvek alapján, *Given–When–Then* formában fogalmazom meg. Az alábbi esetek a tényleges felhasználói felületének elemeihez (gombok, csúszkák, céloztálya-kereső) igazodnak, és kézi (manuális) végrehajtásra tervezettek.

- **Kép betöltése a „Load Image” gombbal**

AS A: felhasználó I WANT TO: egy képet betölteni a vizsgálathoz

GIVEN: az alkalmazás elindítva, még nincs kép betöltve

WHEN: a „Load Image” gombra kattint, majd egy érvényes képfájlt választ ki

THEN: az „Original” képmezőben megjelenik a beolvasott,  $224 \times 224$ -re vágott kép, és alatta a top-3 predikció és konfidencia frissül

- **Modell váltása betöltött kép mellett**

AS A: felhasználó I WANT TO: összehasonlítani, hogy más architektúra hogyan látja a képet

GIVEN: egy kép már be van töltve, az „Original” predikciós panel ki van töltve

WHEN: a modell-választó legördülőben „MobileNetV2”-t választ „ResNet50” helyett

THEN: az „Original” predikciók újraszámolódnak az új modellel; ha korábban már futott támadás, az „Adversarial” predikciók is frissülnek

- **Non-targeted támadás végrehajtása**

AS A: felhasználó I WANT TO: általános tévesztést előidézni a modellben

GIVEN: kép betöltve, „Targeted attack” jelölőnégyzet kikapcsolva,  $\epsilon$  és iterációs szám csúszkák beállítva

WHEN: az „Attack” gombra kattint

THEN: a státuszsor „Running...” majd „Attack complete” üzenetet mutat, megjelenik a felnagyított perturbáció és a támadott kép, és az „Adversarial” predikciós panelen az eredeti top-1 osztály konfidenciája jellemzően csökken

- **Targeted támadás célosztály kereséssel**

AS A: felhasználó I WANT TO: egy konkrét osztályra rávenni a modellt

GIVEN: kép betöltve, a célosztály-kereső mezőbe elkezd gépelni egy osztálynevet

WHEN: a szűrt listából kiválaszt egy osztályt, majd az „Attack” gombra kattint

THEN: a „Targeted attack” jelölőnégyzet automatikusan bejelölődik, a státuszsor a kiválasztott osztály nevét mutatja, és a támadás után a célosztály konfidenciája nő

- **Epsilon csúszka élő előnézete FGSM módban**

AS A: felhasználó I WANT TO: interaktívan látni, hogyan változik a perturbáció erőssége

GIVEN: FGSM módszerrel már lefutott egy támadás (a gradiens előjele cache-elve van)

WHEN: az  $\epsilon$  csúszkát új értékre húzza, PGD-re váltás nélkül

THEN: a perturbáció és az adversarial kép új gradiens-számítás *nélkül*, azonnal újraszámolódik a cache-elt előjel alapján

- **Attack gomb kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) támadást indítani kép betöltése előtt

GIVEN: az alkalmazás elindítva, nincs betöltött kép

WHEN: az „Attack” gombra kattint

THEN: egy „No image” információs párbeszédablak jelenik meg, a támadás nem indul el

- **Védekezési szűrő alkalmazása (Blur / JPEG)**

AS A: felhasználó I WANT TO: megvizsgálni, hogy egy egyszerű szűrő semlegesíti-e a zajt

GIVEN: egy sikeres támadás után az „Adversarial” kép elérhető

WHEN: a „Defend (Blur)” vagy „Defend (JPEG)” gombra kattint

THEN: az „Adversarial” képmező a szűrt képre frissül, és a predikciós panel az újraosztályozott (a szűrés után kapott) eredményeket mutatja

- **Defend gomb adversarial kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) védekezést alkalmazni támadás előtt

GIVEN: kép be van töltve, de még nem futott támadás

WHEN: valamelyik „Defend” gombra kattint

THEN: egy „No adversarial” információs párbeszédablak jelenik meg, semmilyen kép nem változik

- **Reset gomb**

AS A: felhasználó I WANT TO: új kísérletet kezdeni ugyanazzal a képpel

GIVEN: egy támadás és esetleg egy védekezés már lefutott

WHEN: a „Reset” gombra kattint

THEN: a perturbáció és az „Adversarial” képmező üresre vált, az „Adversarial” predikciós panel névtelen („—”) állapotba áll vissza, az „Original” kép és predikciói változatlanok maradnak

- **Hibakezelés támadás közben**

AS A: felhasználó I WANT TO: érthető visszajelzést kapni, ha a támadás valamiért meghiúsul

GIVEN: kép betöltve, a háttérszálon futó `_attack_worker` kivételt dob (pl. eszközhiba miatt)

WHEN: az „Attack” gombra kattint

THEN: egy „Attack failed” hibaablak jelenik meg a teljes veremkiíratással, a státuszsor „Attack failed.” szöveget mutat, az alkalmazás nem fagy le

## **5.5. Teszteredmények és értelmezésük**

## **5.6. A tesztelés korlátai**

## 6. Összefoglalás

### 6.1. Elért eredmények

### 6.2. Következtetések

### 6.3. Továbbfejlesztési lehetőségek

## Mellékletek

## Hivatkozások

- [1] Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. arXiv:1412.6572, 2015.
- [2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. arXiv:1706.06083, 2018.
- [3] Martin Fowler. *TestPyramid*. martinowler.com, 2012. <https://martinfowler.com/bliki/TestPyramid.html>
- [4] Gintare Karolina Dziugaite, Zoubin Ghahramani, Daniel M. Roy. *A study of the effect of JPG compression on adversarial images*. arXiv:1608.00853, 2016.